

E-commerce Order & Supply Chain SQL Table Creation Coding

```
-- =====  
-- TABLE: customers  
-- =====  
CREATE TABLE customers (  
    customer_id        VARCHAR(20) NOT NULL PRIMARY KEY,  
    customer_name       VARCHAR(255) NOT NULL,  
    customer_city       VARCHAR(50) NOT NULL,  
    customer_state      VARCHAR(20) NOT NULL,  
    customer_country    VARCHAR(100) NOT NULL  
)  
  
-- =====  
-- TABLE: products  
-- =====  
CREATE TABLE products (  
    product_id         VARCHAR(50) NOT NULL PRIMARY KEY,  
    product_category   VARCHAR(100) NOT NULL,  
    product_weight_g   DECIMAL(10,2) NOT NULL,  
    product_length_cm  DECIMAL(10,2) NOT NULL,  
    product_height_cm  DECIMAL(10,2) NOT NULL,  
    product_width_cm   DECIMAL (10,2) NOT NULL  
)
```

```

-- =====
-- TABLE: orders
-- =====
CREATE TABLE orders (
    ordered_id          VARCHAR(50) NOT NULL PRIMARY KEY,
    customer_id         VARCHAR(50) NOT NULL,
    order_status        VARCHAR(50) NOT NULL,
    order_date          DATE NOT NULL,
    delivery_date       DATE NOT NULL,
    estimated_delivery_date DATE NOT NULL,
    shipping_city        VARCHAR(100) NOT NULL,
    shipping_state       VARCHAR(100) NOT NULL,

    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE
)

-- =====
-- TABLE: order_items
-- =====
CREATE TABLE order_items (
    order_item_id       INTEGER NOT NULL PRIMARY KEY,
    order_id            VARCHAR(50) NOT NULL,
    product_id          VARCHAR(50) NOT NULL,
    price               DECIMAL(10,2) NOT NULL,
    shipping_charges    DECIMAL(10,2),

    CONSTRAINT fk_items_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_items_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE CASCADE,

    CHECK (price >= 0)
)

```

```

-- =====
-- TABLE: payments
-- =====
CREATE TABLE payments (
    payment_id          INTEGER(11) NOT NULL PRIMARY KEY,
    order_id            VARCHAR(50) NOT NULL,
    payment_sequential  INTEGER(11) NOT NULL,
    payment_type        VARCHAR(50) NOT NULL,
    payment_installments INTEGER(11) NOT NULL,
    payment_value       DECIMAL(10,2) NOT NULL

    CONSTRAINT fk_payments_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE,

    CHECK (payment_value >= 0),
    CHECK (payment_installments >= 0)
)

ALTER TABLE orders
ADD COLUMN order_approved_at DATETIME
AFTER order_date;

ALTER TABLE order_items
ADD COLUMN seller_id VARCHAR(50) NOT NULL
AFTER product_id;

```

Istruzioni per il popolamento

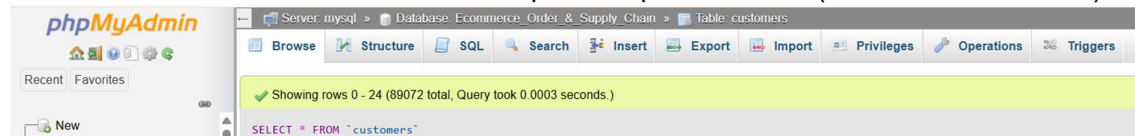
Caricamento massivo (Bulk loading): importazione di grandi quantità di dati da file esterni (es. CSV file), sui quali è già stata effettuata la pulizia.

Ho importato i cinque file CSV elencati di seguito:

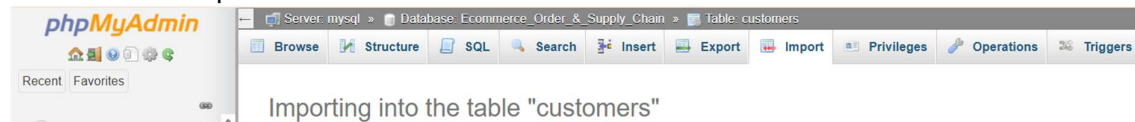
1. df_Customers_NEW.csv
2. dfproductscopy_NEW.csv
3. dforderscopy_transformed.csv
4. dforder_items_transformed.csv
5. dfpayments_transformed.csv

1. In **localhost** su phpMyAdmin, seleziona il database nel quale desideri importare il file CSV (es. [Database: Ecommerce_Order_& Supply_Chain](#)).

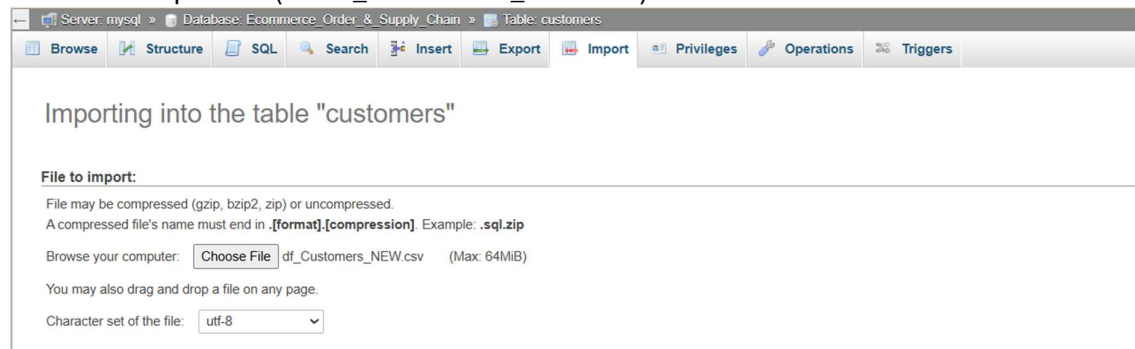
2. Seleziona la tabella di destinazione specifica per il file CSV (es. [Table: customers](#))



3. Fai clic in Import.



4. Fai clic in '**Scegli file o Choose File**', sfoglia le cartelle del computer e seleziona il file da importare (es. df_Customers_NEW.csv)



5. Riempi come faccio io nella foto.

- In **Partial import**: Uncheck/Deseleziona/Togli la spunta Allow the interruption of an import in case the script detects it is close to the PHP timeout limit. (This might be a good way to import large files, however it can break transactions.). Skip this number of queries (for SQL) starting from the first one: 1.
- In **Other options**: Uncheck/Deseleziona/Togli la spunta Enable foreign key checks.
- In **Format**: seleziona CSV (o il tipo di file da importare).
- Clicca su Go/Vai.

Server: mysql » Database: Ecommerce_Order_& Supply_Chain » Table: customers

Browse Structure SQL Search Insert Export Import Privileges Operations Triggers

Importing into the table "customers"

File to import:

File may be compressed (gzip, bzip2, zip) or uncompressed.
A compressed file's name must end in **[format].[compression]**. Example: **.sql.zip**

Browse your computer: df_Customers_NEW.csv (Max: 64MiB)

You may also drag and drop a file on any page.

Character set of the file:

Partial import:

☐ Allow the interruption of an import in case the script detects it is close to the PHP timeout limit. (This might be a good way to import large files, however it can break transactions.)

Skip this number of queries (for SQL) starting from the first one:

Other options:

☐ Enable foreign key checks

Format:

Note: If the file contains multiple tables, they will be combined into one.

Format-specific options:

☐ Update data when duplicate keys found on import (add ON DUPLICATE KEY UPDATE)

Columns separated with:

Columns enclosed with:

Columns escaped with:

Lines terminated with:

Import these many number of rows (optional):

Column names:

☐ Do not abort on INSERT error

- Al termine dell'importazione, un messaggio confermerà il successo dell'operazione; in caso contrario, verrà segnalato un errore.